



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 3: Programación secuencial en Java

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 3 de septiembre de 2026

Índice general

Índice de figuras	III
1. Conversión de temperatura	1
2. Área y perímetro de un rectángulo	2
3. Promedio y aprobación	3
4. Clasificación de un número	5
5. Calculadora básica	6
6. Tabla de multiplicar	7
7. Suma y promedio de varios números	8
8. Análisis de un arreglo	10
9. Búsqueda secuencial	12
10. Sistema de calificaciones	14
Referencias	16

Índice de figuras

1.1. Captura de la conversión de temperatura.	1
2.1. Captura del área y perímetro de un rectángulo.	2
3.1. Captura del promedio y la aprobación.	4
4.1. Captura de la clasificación de un número.	5
5.1. Captura de la calculadora básica.	6
6.1. Captura de la tabla de multiplicar.	7
7.1. Captura de la suma y promedio de varios números.	9
8.1. Captura del análisis de un arreglo.	11
9.1. Captura de la búsqueda secuencial.	13
10.1. Captura del sistema de calificaciones.	15

1. Conversión de temperatura

Solicita una temperatura en grados Celsius y conviértela a Fahrenheit.

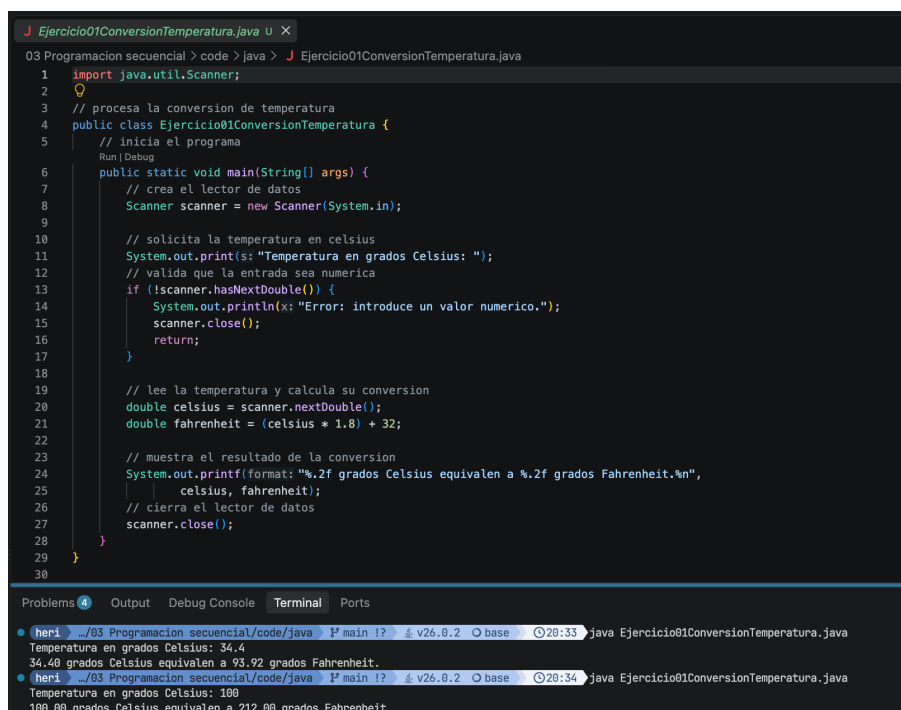
Entradas. Una temperatura real expresada en grados Celsius.

Procesamiento. Se multiplica la temperatura Celsius por 1.8 y se suma 32 para obtener la temperatura Fahrenheit.

Salidas. La temperatura original y en grados Fahrenheit.

Estructuras de control utilizadas. Secuencia de instrucciones (leer la entrada, realizar la operación de conversión y mostrar el resultado) y una condicional if-else para validar que la entrada tenga formato numérico.

Conceptos. Una variable almacena un dato que puede utilizarse durante el procesamiento. La entrada se obtiene mediante Scanner; la operación aritmética aplica la fórmula de conversión; y la salida comunica el resultado.



```
J Ejercicio01ConversionTemperatura.java U X
03 Programacion secuencial > code > java > J Ejercicio01ConversionTemperatura.java
1  import java.util.Scanner;
2
3  // procesa la conversion de temperatura
4  public class Ejercicio01ConversionTemperatura {
5      // inicia el programa
6      public static void main(String[] args) {
7          // crea el lector de datos
8          Scanner scanner = new Scanner(System.in);
9
10         // solicita la temperatura en celsius
11         System.out.print(s: "Temperatura en grados Celsius: ");
12         // valida que la entrada sea numerica
13         if (!scanner.hasNextDouble()) {
14             System.out.println(x: "Error: introduce un valor numerico.");
15             scanner.close();
16             return;
17         }
18
19         // lee la temperatura y calcula su conversion
20         double celsius = scanner.nextDouble();
21         double fahrenheit = (celsius * 1.8) + 32;
22
23         // muestra el resultado de la conversion
24         System.out.printf(format: "%.2f grados Celsius equivalen a %.2f grados Fahrenheit.\n",
25             celsius, fahrenheit);
26         // cierra el lector de datos
27         scanner.close();
28     }
29 }
30

Problems 4 Output Debug Console Terminal Ports
theri > ~/83 Programacion secuencial/code/java P main !? v26.0.2 O base 20:33 java Ejercicio01ConversionTemperatura.java
Temperatura en grados Celsius: 34.4
34.40 grados Celsius equivalen a 93.92 grados Fahrenheit.
theri > ~/83 Programacion secuencial/code/java P main !? v26.0.2 O base 20:34 java Ejercicio01ConversionTemperatura.java
Temperatura en grados Celsius: 100
100.00 grados Celsius equivalen a 212.00 grados Fahrenheit.
```

Figura 1.1: Captura de la conversión de temperatura.

2. Área y perímetro de un rectángulo

Solicita la base y la altura de un rectángulo. Calcula y muestra su área y su perímetro.

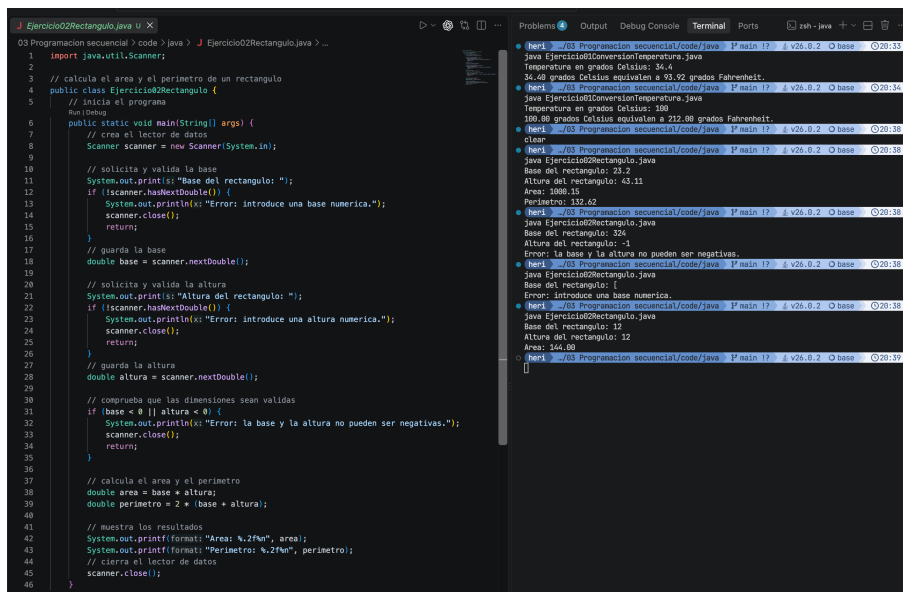
Entradas. La base y la altura del rectángulo, expresadas como números reales no negativos.

Procesamiento. Se multiplican la base y la altura para calcular el área; después se duplica la suma de ambas dimensiones para calcular el perímetro.

Salidas. El área y el perímetro calculados, con dos decimales.

Estructuras de control utilizadas. Secuencia de instrucciones y una condicional para validar el tipo de dato y que las dimensiones no sean negativas.

Conceptos. La secuencia ejecuta las operaciones en orden. La reutilización de datos permite usar la misma base y altura en dos fórmulas distintas sin volver a solicitarlas.



```
03 Programacion secuencial > code > java > J Ejercicio02Rectangulo.java > ...
1  import java.util.Scanner;
2
3  // calcula el area y el perimetro de un rectangulo
4  public class Ejercicio02Rectangulo {
5      // inicia el programa
6      // comienza
7      public static void main(String[] args) {
8          // crea el lector de datos
9          Scanner scanner = new Scanner(System.in);
10
11         // solicita y valida la base
12         System.out.print("Base del rectangulo: ");
13         if (!scanner.hasNextDouble()) {
14             System.out.println("Error: introduce una base numerica.");
15             scanner.close();
16             return;
17         }
18         // guarda la base
19         double base = scanner.nextDouble();
20
21         // solicita y valida la altura
22         System.out.print("Altura del rectangulo: ");
23         if (!scanner.hasNextDouble()) {
24             System.out.println("Error: introduce una altura numerica.");
25             scanner.close();
26             return;
27         }
28         // guarda la altura
29         double altura = scanner.nextDouble();
30
31         // comprueba que las dimensiones sean validas
32         if (base < 0 || altura < 0) {
33             System.out.println("Error: la base y la altura no pueden ser negativas.");
34             scanner.close();
35             return;
36         }
37
38         // calcula el area y el perimetro
39         double area = base * altura;
40         double perimetro = 2 * (base + altura);
41
42         // muestra los resultados
43         System.out.printf("Area: %.2fm", area);
44         System.out.printf("Perimetro: %.2fm", perimetro);
45         // cierra el lector de datos
46         scanner.close();
47     }
48 }
```

Terminal Output:

```
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:33
java Ejercicio02ConversionTemperatura.java
Temperatura en grados Celsius: 34.4
34.40 grados Celsius equivalen a 93.92 grados Fahrenheit.
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:34
java Ejercicio02ConversionTemperatura.java
Temperatura en grados Celsius: 100
100.00 grados Celsius equivalen a 212.00 grados Fahrenheit.
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:36
clear
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:38
java Ejercicio02Rectangulo.java
Base del rectangulo: 23.2
Altura del rectangulo: 43.11
Area: 1000.15
Perimetro: 132.62
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:38
java Ejercicio02Rectangulo.java
Base del rectangulo: 324
Altura del rectangulo: -1
Error: la base y la altura no pueden ser negativas.
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:38
java Ejercicio02Rectangulo.java
Base del rectangulo: 1
Error: introduce una base numerica.
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:38
java Ejercicio02Rectangulo.java
Base del rectangulo: 12
Altura del rectangulo: 12
Area: 144.00
heri ~/03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O20:39
J
```

Figura 2.1: Captura del área y perímetro de un rectángulo.

3. Promedio y aprobación

Solicita tres calificaciones, calcula su promedio e indica si el estudiante está aprobado cuando el promedio es mayor o igual que 6, o reprobado cuando es menor que 6.

Entradas. Tres calificaciones reales en el intervalo de 0 a 10.

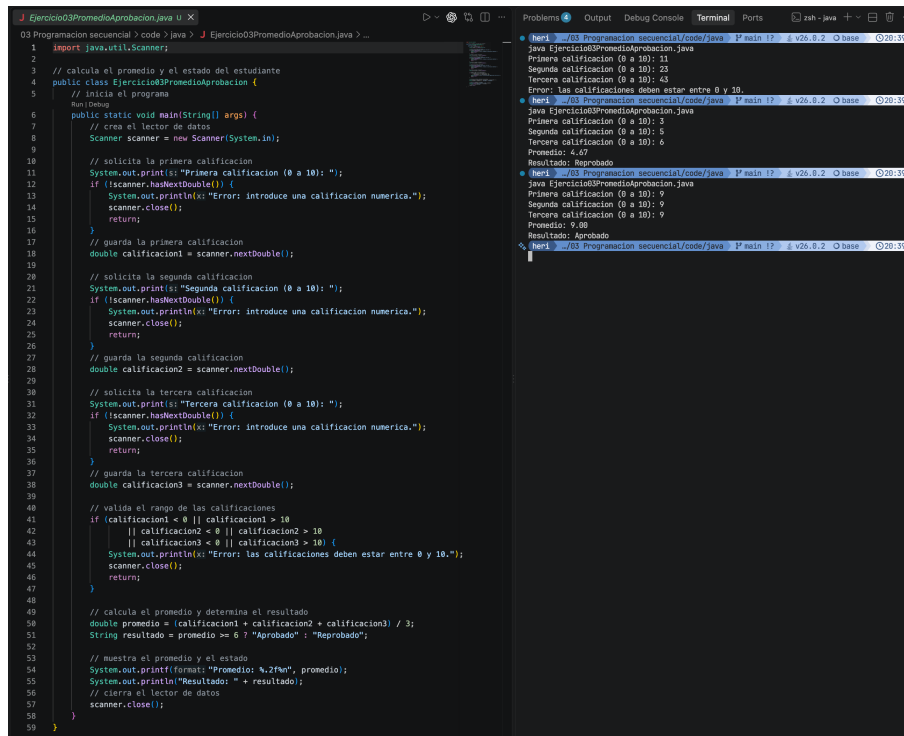
Procesamiento. Se suman las tres calificaciones y se divide el resultado entre 3. Después se compara el promedio con el valor mínimo de aprobación, 6.

Salidas. El promedio calculado y el estado del estudiante: aprobado o reprobado.

Estructuras de control utilizadas. Secuencia de instrucciones y una condicional para seleccionar el estado; también se usa una condicional para validar el intervalo de las calificaciones.

Conceptos. La entrada proporciona los datos que serán procesados. El promedio es una operación aritmética y la estructura if-else permite tomar una decisión según una condición.

Ejercicio 3. Promedio y aprobación



```
03 Programacion secuencial > code > java > J Ejercicio03PromedioAprobacion.java > ...
1 import java.util.Scanner;
2
3 // calcula el promedio y el estado del estudiante
4 public class Ejercicio03PromedioAprobacion {
5     // inicia el programa
6     public static void main(String[] args) {
7         // crea el lector de datos
8         Scanner scanner = new Scanner(System.in);
9
10        // solicita la primera calificación
11        System.out.println("Primera calificación (0 a 10): ");
12        if (!scanner.hasNextDouble()) {
13            System.out.println("Error: introduce una calificación numerica.");
14            scanner.close();
15            return;
16        }
17        // guarda la primera calificación
18        double calificacion1 = scanner.nextDouble();
19
20        // solicita la segunda calificación
21        System.out.println("Segunda calificación (0 a 10): ");
22        if (!scanner.hasNextDouble()) {
23            System.out.println("Error: introduce una calificación numerica.");
24            scanner.close();
25            return;
26        }
27        // guarda la segunda calificación
28        double calificacion2 = scanner.nextDouble();
29
30        // solicita la tercera calificación
31        System.out.println("Tercera calificación (0 a 10): ");
32        if (!scanner.hasNextDouble()) {
33            System.out.println("Error: introduce una calificación numerica.");
34            scanner.close();
35            return;
36        }
37        // guarda la tercera calificación
38        double calificacion3 = scanner.nextDouble();
39
40        // valida el rango de las calificaciones
41        if (calificacion1 < 0 || calificacion1 > 10 ||
42            calificacion2 < 0 || calificacion2 > 10 ||
43            calificacion3 < 0 || calificacion3 > 10) {
44            System.out.println("Error: las calificaciones deben estar entre 0 y 10.");
45            scanner.close();
46            return;
47        }
48
49        // calcula el promedio y determina el resultado
50        double promedio = (calificacion1 + calificacion2 + calificacion3) / 3;
51        String resultado = promedio >= 6 ? "Aprobado" : "Reprobado";
52
53        // muestra el promedio y el estado
54        System.out.printf("Promedio: %.2f\n", promedio);
55        System.out.println("Resultado: " + resultado);
56        // cierra el lector de datos
57        scanner.close();
58    }
59 }
```

Terminal Output:

```
java Ejercicio03PromedioAprobacion.java
Primera calificación (0 a 10): 11
Segunda calificación (0 a 10): 22
Tercera calificación (0 a 10): 43
Error: las calificaciones deben estar entre 0 y 10.
java Ejercicio03PromedioAprobacion.java
Primera calificación (0 a 10): 3
Segunda calificación (0 a 10): 5
Tercera calificación (0 a 10): 6
Promedio: 4.67
Resultado: Reprobado
java Ejercicio03PromedioAprobacion.java
Primera calificación (0 a 10): 9
Segunda calificación (0 a 10): 9
Tercera calificación (0 a 10): 9
Promedio: 9.00
Resultado: Aprobado
```

Figura 3.1: Captura del promedio y la aprobación.

4. Clasificación de un número

Solicita un número entero y determina si es positivo, negativo o cero, y si es par o impar. Considera que el cero es un número par.

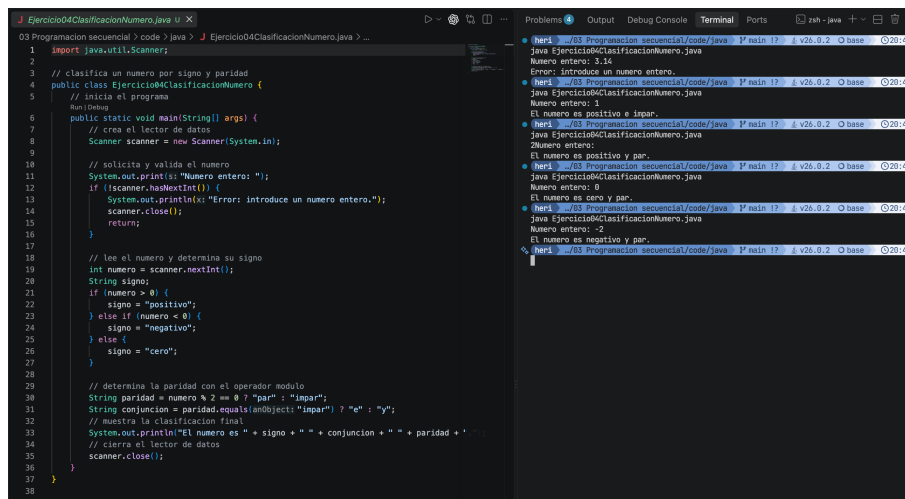
Entradas. Un número entero.

Procesamiento. Se compara el número con cero para clasificar su signo. Después se calcula el residuo de la división entre 2; si el residuo es cero, el número es par, incluido el caso del cero.

Salidas. Una descripción que indique el signo y la paridad del número.

Estructuras de control utilizadas. Condicional para el signo y una condicional para la paridad.

Conceptos. Los operadores comparan el número con cero. El operador módulo obtiene el residuo de una división entera y permite distinguir entre números pares e impares.



```
03 Programacion secuencial > code > java > J Ejercicio04ClasificacionNumero.java > ...
1 import java.util.Scanner;
2
3 // clasifica un numero por signo y paridad
4 public class Ejercicio04ClasificacionNumero {
5     // inicia el programa
6
7     // solicita y valida el numero
8     public static void main(String[] args) {
9         // crea el lector de datos
10        Scanner scanner = new Scanner(System.in);
11
12        // lee el numero y determina su signo
13        int numero = scanner.nextInt();
14        String signo;
15        if (numero > 0) {
16            signo = "positivo";
17        } else if (numero < 0) {
18            signo = "negativo";
19        } else {
20            signo = "cero";
21        }
22
23        // determina la paridad con el operador modulo
24        String paridad = numero % 2 == 0 ? "par" : "impar";
25        String conjuncion = paridad.equals("impar") ? "e" : "y";
26        // muestra la clasificacion final
27        System.out.println("El numero es " + signo + " " + conjuncion + " " + paridad + ".");
28        // cierra el lector de datos
29        scanner.close();
30    }
31 }
```

Problems Output Debug Console Terminal Ports

- Java Ejercicio04ClasificacionNumero.java
Numero entero: 3.14
Error: Introduce un numero entero.
- Java Ejercicio04ClasificacionNumero.java
Numero entero: 1
El numero es positivo e impar.
- Java Ejercicio04ClasificacionNumero.java
Numero entero: 0
El numero es positivo y par.
- Java Ejercicio04ClasificacionNumero.java
Numero entero: 0
El numero es cero y par.
- Java Ejercicio04ClasificacionNumero.java
Numero entero: -2
El numero es negativo y par.

Figura 4.1: Captura de la clasificación de un número.

5. Calculadora básica

Solicita dos números y una operación: 1) suma, 2) resta, 3) multiplicación o 4) división. Realiza la operación seleccionada y muestra el resultado; en la división, verifica que el segundo número no sea cero.

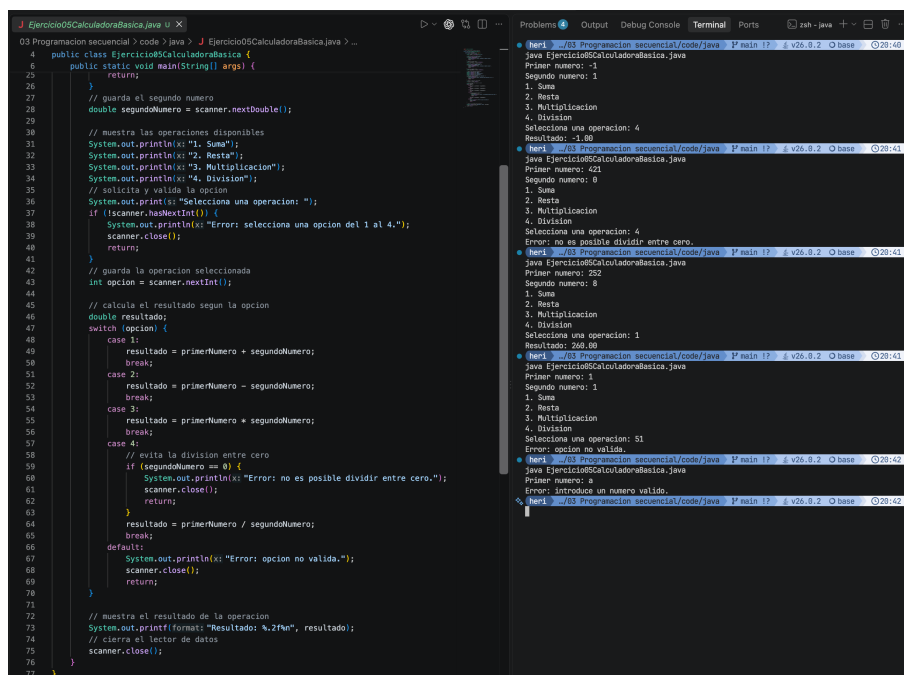
Entradas. Dos números reales y una opción entera entre 1 y 4.

Procesamiento. La opción seleccionada determina la operación. Antes de dividir se comprueba que el segundo número sea distinto de cero.

Salidas. El resultado de la suma, resta, multiplicación o división, o un mensaje claro cuando la opción o el divisor no son válidos.

Estructuras de control utilizadas. La estructura `switch` selecciona la operación y una condicional valida la división entre cero y las opciones fuera del intervalo permitido.

Conceptos. `switch` permite elegir entre varios casos. La validación evita realizar una operación no definida y las operaciones producen el resultado.



```
03 Programacion secuencial > code > java > J Ejercicio05CalculadoraBasica.java > ...
4 public class Ejercicio05CalculadoraBasica {
5     public static void main(String[] args) {
6         return;
7     }
8 }
9
10 // guarda el segundo numero
11 double segundoNumero = scanner.nextDouble();
12
13 // muestra las operaciones disponibles
14 System.out.println("1. Suma");
15 System.out.println("2. Resta");
16 System.out.println("3. Multiplicacion");
17 System.out.println("4. Division");
18 // solicita y valida la opcion
19 System.out.print("Selecciona una operacion: ");
20 if (!scanner.hasNextInt()) {
21     System.out.println("Error: selecciona una opcion del 1 al 4.");
22     scanner.close();
23     return;
24 }
25 // guarda la operacion seleccionada
26 int opcion = scanner.nextInt();
27
28 // calcula el resultado segun la opcion
29 double resultado;
30 switch (opcion) {
31     case 1:
32         resultado = primerNumero + segundoNumero;
33         break;
34     case 2:
35         resultado = primerNumero - segundoNumero;
36         break;
37     case 3:
38         resultado = primerNumero * segundoNumero;
39         break;
40     case 4:
41         // evita la division entre cero
42         if (segundoNumero == 0) {
43             System.out.println("Error: no es posible dividir entre cero.");
44             scanner.close();
45             return;
46         }
47         resultado = primerNumero / segundoNumero;
48         break;
49     default:
50         System.out.println("Error: opcion no valida.");
51         scanner.close();
52         return;
53 }
54
55 // muestra el resultado de la operacion
56 System.out.printf("Resultado: %.2f\n", resultado);
57 // cierra el lector de datos
58 scanner.close();
59 }
60 }
```

```
heri ~/3 Programacion secuencial/codigo/java P main 17 6 v26.0.2 O base O20:40
java Ejercicio05CalculadoraBasica.java
Primer numero: -1
Segundo numero: 1
1. Suma
2. Resta
3. Multiplicacion
4. Division
Selecciona una operacion: 4
Resultado: -1.00

heri ~/3 Programacion secuencial/codigo/java P main 17 6 v26.0.2 O base O20:41
java Ejercicio05CalculadoraBasica.java
Primer numero: 421
Segundo numero: 0
1. Suma
2. Resta
3. Multiplicacion
4. Division
Selecciona una operacion: 4
Error: no es posible dividir entre cero.

heri ~/3 Programacion secuencial/codigo/java P main 17 6 v26.0.2 O base O20:41
java Ejercicio05CalculadoraBasica.java
Primer numero: 252
Segundo numero: 8
1. Suma
2. Resta
3. Multiplicacion
4. Division
Selecciona una operacion: 1
Resultado: 260.00

heri ~/3 Programacion secuencial/codigo/java P main 17 6 v26.0.2 O base O20:41
java Ejercicio05CalculadoraBasica.java
Primer numero: 1
Segundo numero: 1
1. Suma
2. Resta
3. Multiplicacion
4. Division
Selecciona una operacion: 51
Error: opcion no valida.

heri ~/3 Programacion secuencial/codigo/java P main 17 6 v26.0.2 O base O20:42
java Ejercicio05CalculadoraBasica.java
Primer numero: 4
Error: introduce un numero valido.
```

Figura 5.1: Captura de la calculadora básica.

6. Tabla de multiplicar

Solicita un número entero y muestra su tabla de multiplicar del 1 al 10.

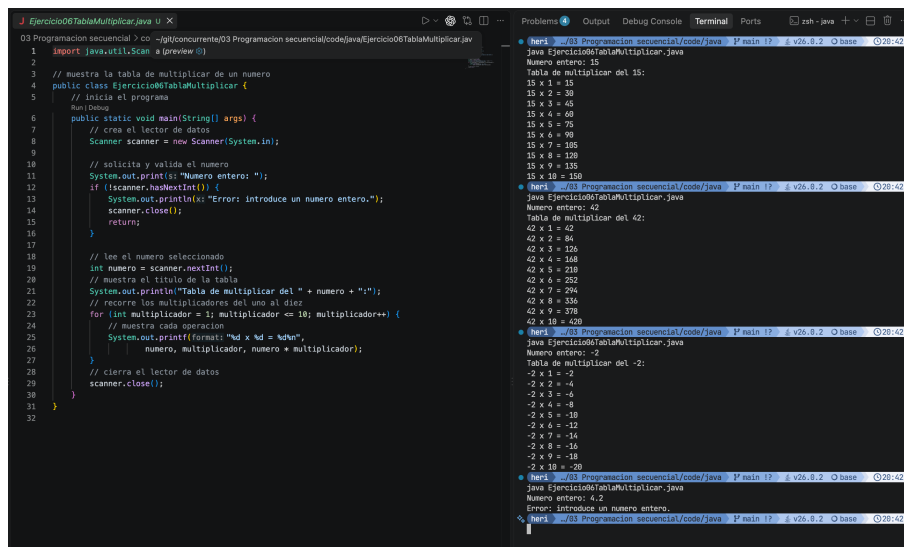
Entradas. Un número entero.

Procesamiento. Se multiplica el número recibido por cada multiplicador entero desde 1 hasta 10.

Salidas. Diez líneas con las multiplicaciones y sus resultados.

Estructuras de control utilizadas. Un ciclo for con un contador que inicia en 1, termina en 10 y aumenta de uno en uno.

Conceptos. El ciclo for automatiza una operación repetitiva. El contador identifica cada multiplicador y la salida repetitiva muestra cada resultado en una línea diferente.



The image shows a screenshot of an IDE with two panels. The left panel displays the source code for a Java program named `Ejercicio06TablaMultiplicar.java`. The code uses a `Scanner` to read an integer from the user, then uses a `for` loop to calculate and print the multiplication table for that number. The right panel shows the output of the program for three different inputs: 15, 42, and -2. Each input results in a table of 10 multiplication facts.

```
03 Programacion secuencial > cd -jg/concurrente/03 Programacion secuencial/code/java/Ejercicio06TablaMultiplicar.jav
1  import java.util.Scanner;
2
3  // muestra la tabla de multiplicar de un numero
4  public class Ejercicio06TablaMultiplicar {
5      // inicia el programa
6      public static void main(String[] args) {
7          // crea el lector de datos
8          Scanner scanner = new Scanner(System.in);
9
10         // solicita y valida el numero
11         System.out.println("Numero entero: ");
12         if (!scanner.hasNextInt()) {
13             System.out.println("Error: introduce un numero entero.");
14             scanner.close();
15             return;
16         }
17
18         // lee el numero seleccionado
19         int numero = scanner.nextInt();
20
21         // muestra el titulo de la tabla
22         System.out.println("Tabla de multiplicar del " + numero + ":");
23         // recorre los multiplicadores del uno al diez
24         for (int multiplicador = 1; multiplicador <= 10; multiplicador++) {
25             // muestra cada operacion
26             System.out.printf("%d x %d = %d\n", numero, multiplicador, numero * multiplicador);
27         }
28         // cierra el lector de datos
29         scanner.close();
30     }
31 }
32
```

Output 1 (Input: 15):

```
Java Ejercicio06TablaMultiplicar.java
Numero entero: 15
Tabla de multiplicar del 15:
15 x 1 = 15
15 x 2 = 30
15 x 3 = 45
15 x 4 = 60
15 x 5 = 75
15 x 6 = 90
15 x 7 = 105
15 x 8 = 120
15 x 9 = 135
15 x 10 = 150
```

Output 2 (Input: 42):

```
Java Ejercicio06TablaMultiplicar.java
Numero entero: 42
Tabla de multiplicar del 42:
42 x 1 = 42
42 x 2 = 84
42 x 3 = 126
42 x 4 = 168
42 x 5 = 210
42 x 6 = 252
42 x 7 = 294
42 x 8 = 336
42 x 9 = 378
42 x 10 = 420
```

Output 3 (Input: -2):

```
Java Ejercicio06TablaMultiplicar.java
Numero entero: -2
Tabla de multiplicar del -2:
-2 x 1 = -2
-2 x 2 = -4
-2 x 3 = -6
-2 x 4 = -8
-2 x 5 = -10
-2 x 6 = -12
-2 x 7 = -14
-2 x 8 = -16
-2 x 9 = -18
-2 x 10 = -20
```

Error output (Input: 4.2):

```
Java Ejercicio06TablaMultiplicar.java
Numero entero: 4.2
Error: introduce un numero entero.
```

Figura 6.1: Captura de la tabla de multiplicar.

7. Suma y promedio de varios números

Solicita cuántos números se introducirán. Después, lee los números y calcula la suma, el promedio, el número mayor y el número menor. Valida que la cantidad de números sea mayor que cero.

Entradas. Una cantidad entera mayor que cero y esa cantidad de números reales.

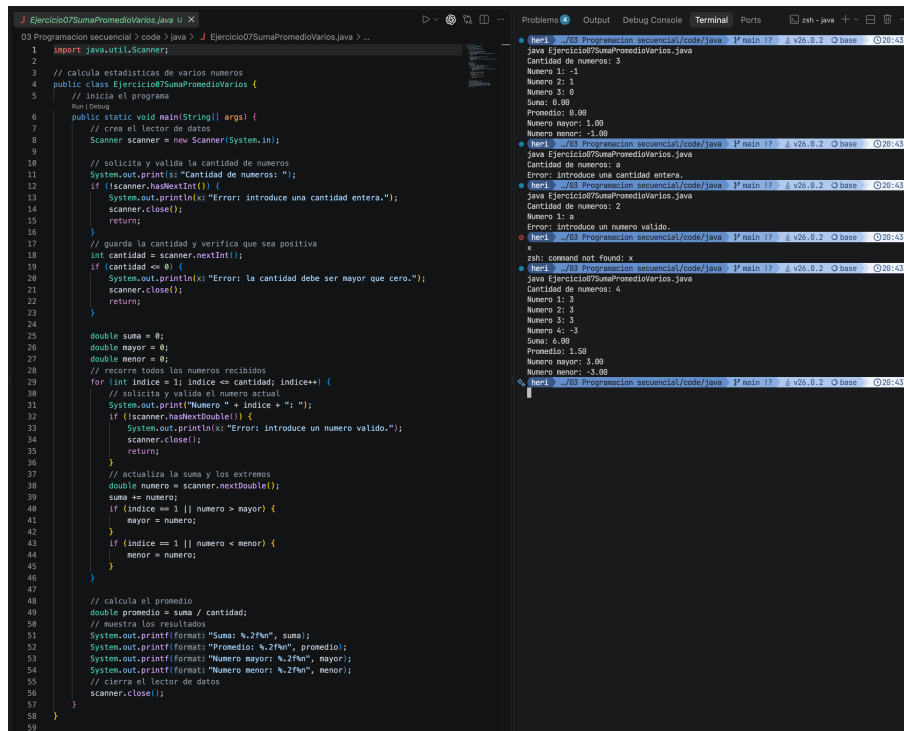
Procesamiento. Se recorre cada número, se acumula la suma y se comparan los valores para actualizar el mayor y el menor. Al finalizar se divide la suma entre la cantidad para obtener el promedio.

Salidas. La suma, el promedio, el número mayor y el número menor.

Estructuras de control utilizadas. Un ciclo for para leer los datos y condicionales para inicializar y actualizar los extremos. También se usa una condicional para validar la cantidad.

Conceptos. Un acumulador conserva la suma parcial. Las comparaciones permiten obtener extremos y la validación evita una cantidad cero que produciría una división inválida.

Ejercicio 7. Suma y promedio de varios números



```
03 Programacion secuencial > code > java > J Ejercicio7SumaPromedioVarios.java ...
1 import java.util.Scanner;
2
3 // calcula estadísticas de varios números
4 public class Ejercicio7SumaPromedioVarios {
5     // inicia el programa
6     public static void main(String[] args) {
7         // crea el lector de datos
8         Scanner scanner = new Scanner(System.in);
9
10        // solicita y valida la cantidad de números
11        System.out.println("Cantidad de números: ");
12        if (!scanner.hasNextInt()) {
13            System.out.println("Error: introduce una cantidad entera.");
14            scanner.close();
15            return;
16        }
17        // guarda la cantidad y verifica que sea positiva
18        int cantidad = scanner.nextInt();
19        if (cantidad <= 0) {
20            System.out.println("Error: la cantidad debe ser mayor que cero.");
21            scanner.close();
22            return;
23        }
24
25        double suma = 0;
26        double mayor = 0;
27        double menor = 0;
28        // recorre todos los números recibidos
29        for (int indice = 1; indice <= cantidad; indice++) {
30            // solicita y valida el número actual
31            System.out.print("Número " + indice + ": ");
32            if (!scanner.hasNextDouble()) {
33                System.out.println("Error: introduce un número válido.");
34                scanner.close();
35                return;
36            }
37            // actualiza la suma y los extremos
38            double numero = scanner.nextDouble();
39            suma += numero;
40            if (indice == 1 || numero > mayor) {
41                mayor = numero;
42            }
43            if (indice == 1 || numero < menor) {
44                menor = numero;
45            }
46        }
47
48        // calcula el promedio
49        double promedio = suma / cantidad;
50        // muestra los resultados
51        System.out.printf("Suma: %.2f\n", suma);
52        System.out.printf("Promedio: %.2f\n", promedio);
53        System.out.printf("Número mayor: %.2f\n", mayor);
54        System.out.printf("Número menor: %.2f\n", menor);
55        // cierra el lector de datos
56        scanner.close();
57    }
58 }
59
```

```
heri_@/03 Programacion secuencial/code/java P main 17 6 v26.0.2 O base O20:43
java Ejercicio7SumaPromedioVarios.java
Cantidad de números: 3
Número 1: -1
Número 2: 1
Número 3: 0
Suma: 0.00
Promedio: 0.00
Número mayor: 1.00
Número menor: -1.00

heri_@/03 Programacion secuencial/code/java P main 17 6 v26.0.2 O base O20:43
java Ejercicio7SumaPromedioVarios.java
Cantidad de números: 4
Error: introduce una cantidad entera.

heri_@/03 Programacion secuencial/code/java P main 17 6 v26.0.2 O base O20:43
java Ejercicio7SumaPromedioVarios.java
Cantidad de números: 2
Número 1: 4
Error: introduce un número válido.

heri_@/03 Programacion secuencial/code/java P main 17 6 v26.0.2 O base O20:43
x
zsh: command not found: x

heri_@/03 Programacion secuencial/code/java P main 17 6 v26.0.2 O base O20:43
java Ejercicio7SumaPromedioVarios.java
Cantidad de números: 4
Número 1: 3
Número 2: 1
Número 3: 3
Número 4: -3
Suma: 4.00
Promedio: 1.00
Número mayor: 3.00
Número menor: -3.00

```

Figura 7.1: Captura de la suma y promedio de varios números.

8. Análisis de un arreglo

Solicita el tamaño de un arreglo y sus elementos. Al finalizar, muestra la suma, el promedio, la cantidad de números positivos, negativos y ceros, y la cantidad de números pares e impares.

Entradas. Un tamaño entero mayor que cero y los elementos enteros del arreglo.

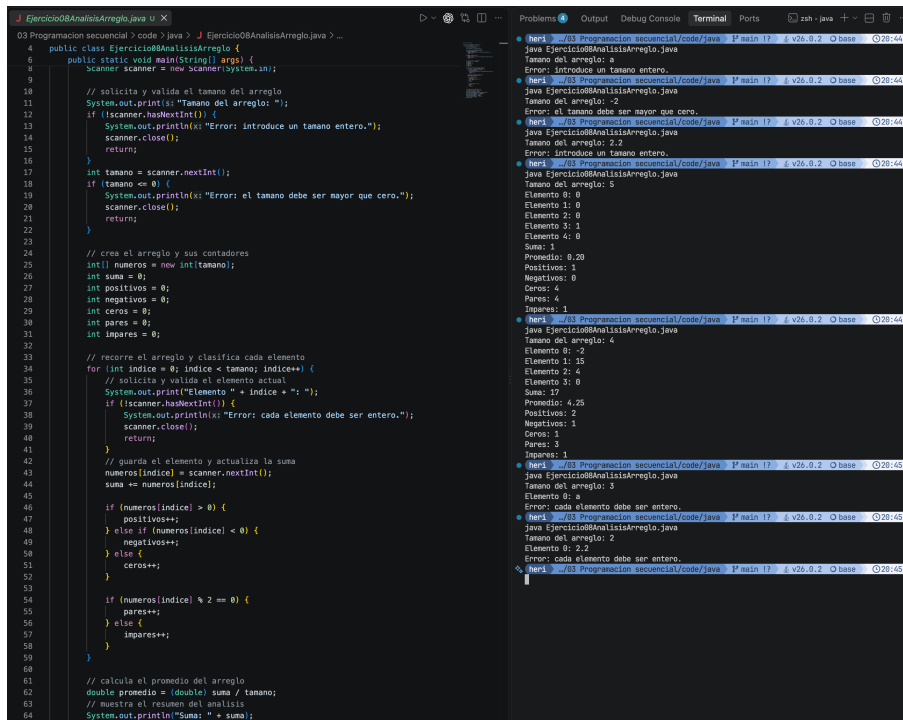
Procesamiento. Se almacenan los elementos en un arreglo y se realiza un recorrido. Durante el recorrido se acumula la suma y se incrementan contadores según el signo y la paridad de cada elemento.

Salidas. La suma, el promedio, las cantidades de positivos, negativos, ceros, pares e impares.

Estructuras de control utilizadas. Un ciclo for para recorrer el arreglo y condicionales anidadas para clasificar cada elemento.

Conceptos. Un arreglo almacena varios valores del mismo tipo. Un recorrido visita cada posición mediante un índice. Los contadores registran categorías y el acumulador conserva la suma.

Ejercicio 8. Análisis de un arreglo



The image shows a screenshot of an IDE with a Java file named `Ejercicio08AnalisisArreglo.java`. The code implements a program to analyze an array of integers. It prompts the user for the array size and then for each element. It then calculates and displays statistics: sum, average, number of positive, negative, even, and odd elements.

```
03 Programacion secuencial > code > java > J Ejercicio08AnalisisArreglo.java > ...
4 public class Ejercicio08AnalisisArreglo {
6     public static void main(String[] args) {
8         Scanner scanner = new Scanner(System.in);
9
10        // solicita y valida el tamaño del arreglo
11        System.out.println("Tamaño del arreglo: ");
12        if (!scanner.hasNextInt()) {
13            System.out.println("Error: introduce un tamaño entero.");
14            scanner.close();
15            return;
16        }
17        int tamaño = scanner.nextInt();
18        if (tamaño <= 0) {
19            System.out.println("Error: el tamaño debe ser mayor que cero.");
20            scanner.close();
21            return;
22        }
23
24        // crea el arreglo y sus contadores
25        int[] numeros = new int[tamaño];
26        int suma = 0;
27        int positivos = 0;
28        int negativos = 0;
29        int pares = 0;
30        int impares = 0;
31
32        // recorre el arreglo y clasifica cada elemento
33        for (int indice = 0; indice < tamaño; indice++) {
34            // solicita y valida el elemento actual
35            System.out.print("Elemento " + indice + ": ");
36            if (!scanner.hasNextInt()) {
37                System.out.println("Error: cada elemento debe ser entero.");
38                scanner.close();
39                return;
40            }
41            // guarda el elemento y actualiza la suma
42            numeros[indice] = scanner.nextInt();
43            suma = numeros[indice];
44
45            if (numeros[indice] > 0) {
46                positivos++;
47            } else if (numeros[indice] < 0) {
48                negativos++;
49            } else {
50                pares++;
51            }
52
53            if (numeros[indice] % 2 == 0) {
54                pares++;
55            } else {
56                impares++;
57            }
58        }
59
60        // calcula el promedio del arreglo
61        double promedio = (double) suma / tamaño;
62        // muestra el resumen del análisis
63        System.out.println("Suma: " + suma);
```

The output window shows the following sequence of interactions:

- Run 1: Input "a", Error: introduce un tamaño entero.
- Run 2: Input "2", Error: el tamaño debe ser mayor que cero.
- Run 3: Input "2.2", Error: introduce un tamaño entero.
- Run 4: Input "5", Output: Tamaño del arreglo: 5, Elemento 0: 0, Elemento 1: 0, Elemento 2: 0, Elemento 3: 1, Elemento 4: 0, Suma: 1, Promedio: 0.20, Positivos: 1, Negativos: 0, Pares: 4, Impares: 1.
- Run 5: Input "4", Output: Tamaño del arreglo: 4, Elemento 0: -2, Elemento 1: 15, Elemento 2: 4, Elemento 3: 0, Suma: 17, Promedio: 4.25, Positivos: 2, Negativos: 1, Pares: 1, Impares: 3.
- Run 6: Input "3", Error: cada elemento debe ser entero.
- Run 7: Input "2", Output: Tamaño del arreglo: 2, Elemento 0: 2.2, Error: cada elemento debe ser entero.

Figura 8.1: Captura del análisis de un arreglo.

9. Búsqueda secuencial

Solicita el tamaño de un arreglo, sus elementos y un número que se desea buscar. Recorre el arreglo y muestra las posiciones donde aparece el número, la cantidad total de apariciones y un mensaje si el número no se encuentra. No utiliza métodos de búsqueda proporcionados por Java.

Entradas. Un tamaño entero mayor que cero, los elementos enteros del arreglo y el número entero que se buscará.

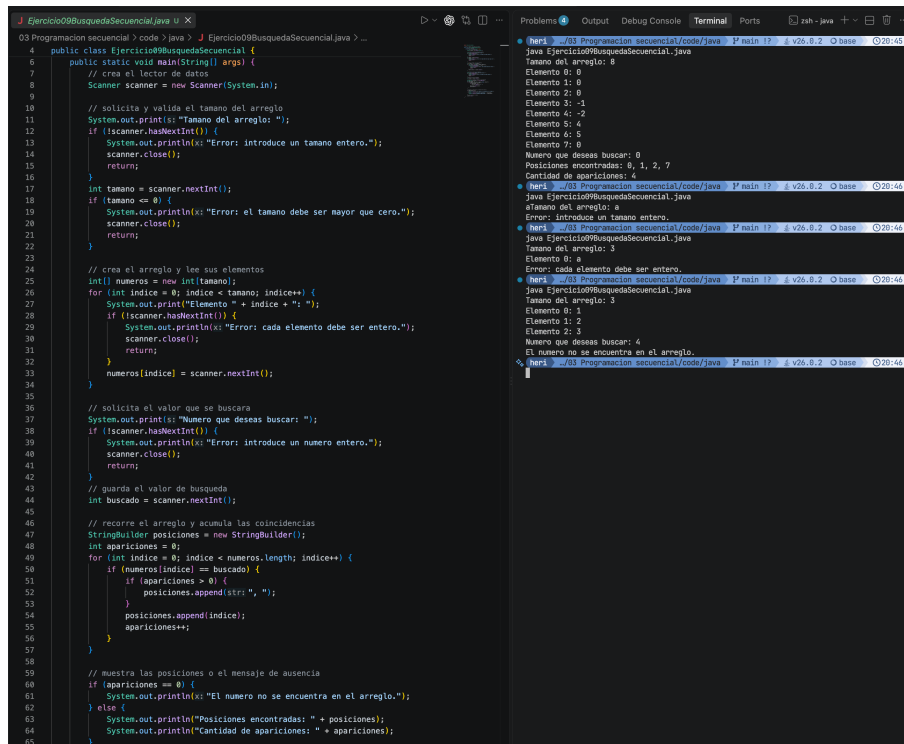
Procesamiento. Se recorre el arreglo desde la primera hasta la última posición. Cada elemento se compara directamente con el valor buscado; cuando coincide, se guarda su índice y se incrementa el contador de apariciones.

Salidas. Los índices donde aparece el valor, el total de apariciones o un mensaje que indique que no se encontró.

Estructuras de control utilizadas. Un ciclo for para el recorrido y condicionales if-else para comparar, contar coincidencias y seleccionar el mensaje final.

Conceptos. La búsqueda secuencial revisa los elementos en orden y usa índices para identificar sus posiciones.

Ejercicio 9. Búsqueda secuencial



The screenshot displays an IDE with a Java file named `Ejercicio09BusquedaSecuencial.java` and its execution output. The code implements a sequential search algorithm. It prompts the user to enter an array size, then the array elements, and finally the search value. It iterates through the array, counting matches and recording their indices. If no matches are found, it prints a message to that effect.

```
public class Ejercicio09BusquedaSecuencial {
    public static void main(String[] args) {
        // crea el lector de datos
        Scanner scanner = new Scanner(System.in);

        // solicita y valida el tamaño del arreglo
        System.out.println("Tamaño del arreglo: ");
        if (!scanner.hasNextInt()) {
            System.out.println("Error: introduce un tamaño entero.");
            scanner.close();
            return;
        }

        int tamaño = scanner.nextInt();
        if (tamaño <= 0) {
            System.out.println("Error: el tamaño debe ser mayor que cero.");
            scanner.close();
            return;
        }

        // crea el arreglo y lee sus elementos
        int[] numeros = new int[tamaño];
        for (int indice = 0; indice < tamaño; indice++) {
            System.out.print("Elemento " + indice + ": ");
            if (!scanner.hasNextInt()) {
                System.out.println("Error: cada elemento debe ser entero.");
                scanner.close();
                return;
            }
            numeros[indice] = scanner.nextInt();
        }

        // solicita el valor que se busca
        System.out.println("Número que deseas buscar: ");
        if (!scanner.hasNextInt()) {
            System.out.println("Error: introduce un número entero.");
            scanner.close();
            return;
        }

        // guarda el valor de búsqueda
        int buscado = scanner.nextInt();

        // recorre el arreglo y acumula las coincidencias
        StringBuilder posiciones = new StringBuilder();
        int apariciones = 0;
        for (int indice = 0; indice < numeros.length; indice++) {
            if (numeros[indice] == buscado) {
                if (apariciones > 0) {
                    posiciones.append(", ");
                }
                posiciones.append(indice);
                apariciones++;
            }
        }

        // muestra las posiciones o el mensaje de ausencia
        if (apariciones == 0) {
            System.out.println("El número no se encuentra en el arreglo.");
        } else {
            System.out.println("Posiciones encontradas: " + posiciones);
            System.out.println("Cantidad de apariciones: " + apariciones);
        }
    }
}
```

The output shows the program's execution with the following data:

- Input: Array size 8, elements [0, 1, 2, 7, 0, 4, 5, 0], search value 0.
- Output: Posiciones encontradas: 0, 1, 2, 7; Cantidad de apariciones: 4.
- Input: Array size a, elements [a], search value a.
- Output: Error: introduce un tamaño entero.
- Input: Array size 3, elements [0, 1, 2], search value 3.
- Output: Error: cada elemento debe ser entero.
- Input: Array size 3, elements [3, 2, 1], search value 3.
- Output: Posiciones encontradas: 0; El número no se encuentra en el arreglo.

Figura 9.1: Captura de la búsqueda secuencial.

10. Sistema de calificaciones

Desarrolla un programa para registrar las calificaciones de varios estudiantes. Debe solicitar la cantidad de estudiantes, solicitar tres calificaciones por estudiante, calcular el promedio individual, indicar si aprobó o reprobó, calcular el promedio general, mostrar el promedio más alto y más bajo, contar aprobados y reprobados, y mostrar el porcentaje de aprobación. Organiza el programa utilizando los métodos `static double calcularPromedio(double c1, double c2, double c3)` y `static boolean estaAprobado(double promedio)`.

Entradas. La cantidad de estudiantes y tres calificaciones reales entre 0 y 10 para cada estudiante.

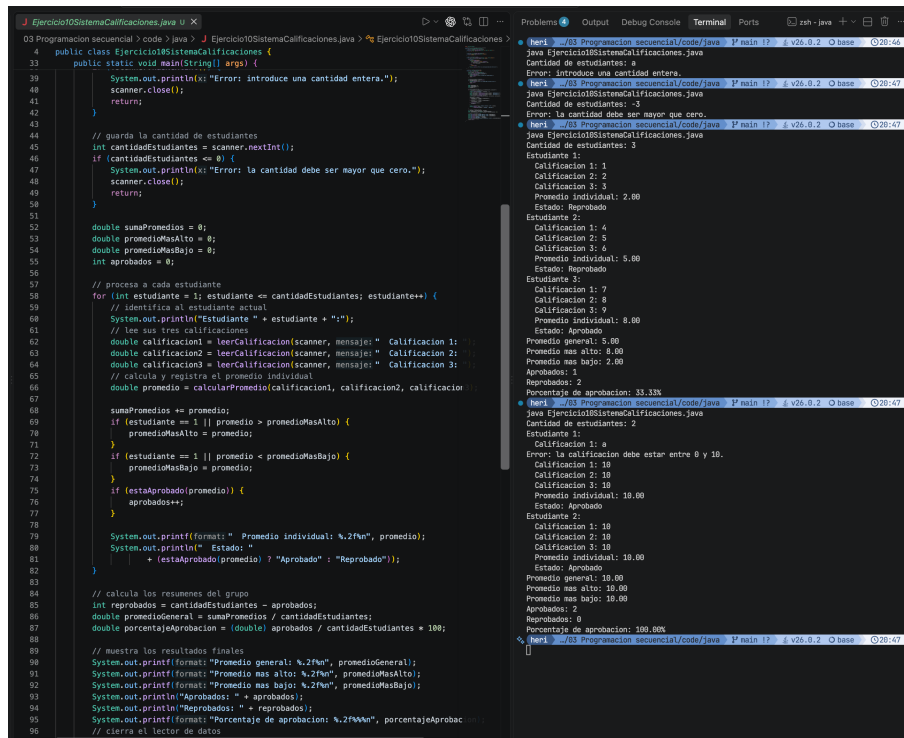
Procesamiento. Para cada estudiante se calcula el promedio individual y se determina su estado. A la vez se acumulan los promedios, se actualizan los valores mayor y menor y se cuentan los aprobados. Al final se calculan el promedio general, los reprobados y el porcentaje de aprobación.

Salidas. El promedio y estado de cada estudiante, el promedio general, los promedios máximo y mínimo, las cantidades de aprobados y reprobados y el porcentaje de aprobación.

Estructuras de control utilizadas. Un ciclo `for` recorre a los estudiantes; condicionales actualizan extremos y contadores; y un ciclo `while` valida cada calificación. Los métodos separan el cálculo del promedio y la decisión de aprobación.

Conceptos. Los acumuladores reúnen promedios y los contadores reúnen categorías.

Ejercicio 10. Sistema de calificaciones



```
03 Programacion secuencial > code > J Ejercicio10SistemaCalificaciones.java > Ejercicio10SistemaCalificaciones.java
4 public class Ejercicio10SistemaCalificaciones {
33 public static void main(String[] args) {
39     System.out.println("Error: introduce una cantidad entera.");
40     scanner.close();
41     return;
42 }
43
44 // guarda la cantidad de estudiantes
45 int cantidadEstudiantes = scanner.nextInt();
46 if (cantidadEstudiantes <= 0) {
47     System.out.println("Error: la cantidad debe ser mayor que cero.");
48     scanner.close();
49     return;
50 }
51
52 double sumaPromedios = 0;
53 double promedioMasAlto = 0;
54 double promedioMasBajo = 0;
55 int aprobados = 0;
56
57 // procesa a cada estudiante
58 for (int estudiante = 1; estudiante <= cantidadEstudiantes; estudiante++) {
59     // identifica al estudiante actual
60     System.out.println("Estudiante " + estudiante + ":");
61     // lee las tres calificaciones
62     double calificacion1 = leerCalificacion(scanner, mensaje: " Calificacion 1: ");
63     double calificacion2 = leerCalificacion(scanner, mensaje: " Calificacion 2: ");
64     double calificacion3 = leerCalificacion(scanner, mensaje: " Calificacion 3: ");
65     // calcula y registra el promedio individual
66     double promedio = calcularPromedio(calificacion1, calificacion2, calificacion3);
67
68     sumaPromedios += promedio;
69     if (estudiante == 1 || promedio > promedioMasAlto) {
70         promedioMasAlto = promedio;
71     }
72     if (estudiante == 1 || promedio < promedioMasBajo) {
73         promedioMasBajo = promedio;
74     }
75     if (estaAprobado(promedio)) {
76         aprobados++;
77     }
78     System.out.printf(format: " Promedio individual: %.2f\n", promedio);
79     System.out.println(" Estado: "
80         + (estaAprobado(promedio) ? "Aprobado" : "Reprobado"));
81 }
82
83 // calcula los resúmenes del grupo
84 int reprobados = cantidadEstudiantes - aprobados;
85 double promedioGeneral = sumaPromedios / cantidadEstudiantes;
86 double porcentajeAprobacion = (double) aprobados / cantidadEstudiantes * 100;
87
88 // muestra los resultados finales
89 System.out.printf(format: "Promedio general: %.2f\n", promedioGeneral);
90 System.out.printf(format: "Promedio mas alto: %.2f\n", promedioMasAlto);
91 System.out.printf(format: "Promedio mas bajo: %.2f\n", promedioMasBajo);
92 System.out.println("Aprobados: " + aprobados);
93 System.out.println("Reprobados: " + reprobados);
94 System.out.printf(format: "Porcentaje de aprobacion: %.2f%%\n", porcentajeAprobacion);
95 // cierra el lector de datos
96 }
```

```
hera /03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O28-46
Java Ejercicio10SistemaCalificaciones.java
Cantidad de estudiantes: 4
Error: introduce una cantidad entera.
hera /03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O28-47
Java Ejercicio10SistemaCalificaciones.java
Cantidad de estudiantes: -3
Error: la cantidad debe ser mayor que cero.
hera /03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O28-47
Java Ejercicio10SistemaCalificaciones.java
Cantidad de estudiantes: 3
Estudiante 1:
Calificacion 1: 1
Calificacion 2: 2
Calificacion 3: 3
Promedio individual: 2.00
Estado: Reprobado
Estudiante 2:
Calificacion 1: 4
Calificacion 2: 5
Calificacion 3: 6
Promedio individual: 5.00
Estado: Reprobado
Estudiante 3:
Calificacion 1: 7
Calificacion 2: 8
Calificacion 3: 9
Promedio individual: 8.00
Estado: Aprobado
Promedio general: 5.00
Promedio mas alto: 8.00
Promedio mas bajo: 2.00
Aprobados: 1
Reprobados: 2
Porcentaje de aprobacion: 33.33%
hera /03 Programacion secuencial/code/java P main 17 4 v26.0.2 O base O28-47
Java Ejercicio10SistemaCalificaciones.java
Cantidad de estudiantes: 2
Estudiante 1:
Calificacion 1: 4
Calificacion 2: 10
Calificacion 3: 10
Promedio individual: 10.00
Estado: Aprobado
Estudiante 2:
Calificacion 1: 10
Calificacion 2: 10
Calificacion 3: 10
Promedio individual: 10.00
Estado: Aprobado
Promedio general: 10.00
Promedio mas alto: 10.00
Promedio mas bajo: 10.00
Aprobados: 2
Reprobados: 0
Porcentaje de aprobacion: 100.00%
```

Figura 10.1: Captura del sistema de calificaciones.

Bibliografía

- [1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Reading, Massachusetts: Addison-Wesley, 1994.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.